

CSCE 608 DATABASE SYSTEMS · MODULE 08 · LECTURE NOTES

Query Optimisation

Choosing a plan, and why the choice is often wrong.

MODULE OUTCOMES

- Explain the two stages of optimisation: rewriting and costing.
- Apply the standard algebraic rewrite rules.
- Explain selectivity estimation and why errors compound.
- Explain dynamic programming join enumeration.
- Diagnose a bad plan and identify the estimation error behind it.

1 Rewriting

Optimisation has two distinct stages. The first applies transformations that are beneficial regardless of the data, so no cost model is consulted. The second chooses among alternatives whose relative cost depends on the data, and that is where the difficulty is.

Rewrite	Example and justification
Predicate pushdown	Move filters as close to the base tables as the semantics allow. Every operator above then processes fewer rows , and no plausible cost model prefers the alternative.
Projection pushdown	Drop columns as soon as they are no longer needed. Narrower tuples mean more rows per page and less memory in every breaker.
Constant folding	WHERE $x > 2 + 3$ becomes WHERE $x > 5$, so the arithmetic is done once rather than per row — and the constant can then be compared against a histogram.
Subquery flattening	Turn a correlated subquery into a join. See below.
Join elimination	If a join's result columns are unused and the join is provably one-to-one via a foreign key, the join can be removed entirely. Common with views and ORMs that join more than the query needs.
Predicate inference	From $a = b$ and $b = 5$, infer $a = 5$. The inferred predicate can then be pushed down to a's table and may enable an index that was not obviously applicable — which surprises people and is genuinely valuable.

SUBQUERY FLATTENING IS OFTEN THE LARGEST SINGLE WIN

A correlated subquery, executed naively, **runs once for every row of the outer query**. A million outer rows means a million executions of the inner query, each with its own setup and its own index lookups.

Flattened into a join, it executes once, and the optimiser can then choose a hash join, reorder it with other joins, and push predicates through it.

This is routinely the difference between seconds and hours, and it is entirely invisible in the query text. The same SQL can be fast on one system and pathological on another purely according to whether that system's optimiser recognises and flattens the pattern.

Not everything can be flattened. NOT EXISTS with complex correlation, aggregates referencing outer columns, and subqueries in the SELECT list with side conditions frequently resist it. When performance matters and a subquery is hot, checking the plan for whether flattening occurred is worth the two minutes.

2 Cost estimation

To compare plans the optimiser must estimate how many rows each operator will produce. **Cardinality estimation is the hard part of query optimisation**, and the rest of the machinery is comparatively straightforward.

Estimate	Formula	Assumption
Equality selectivity	$1 / \text{distinct}(a)$	Uniformity — every distinct value occurs equally often.
Conjunction	$\text{sel}(p) \times \text{sel}(q)$	Independence — the two predicates are statistically unrelated.
Join size	$ R \times S / \max(\text{distinct}_R, \text{distinct}_S)$	Containment — the smaller key set is a subset of the larger.

Real data violates all three, routinely. Naming the assumptions explicitly is what lets you predict where the optimiser will go wrong, which is more useful than any individual tuning trick.

CORRELATION IS THE ASSUMPTION THAT BREAKS MOST OFTEN

Consider WHERE city = 'Dallas' AND state = 'TX'.

The independence assumption computes $\text{sel}(\text{city}) \times \text{sel}(\text{state})$. With 10,000 distinct cities and 50 states, that is $1/10,000 \times 1/50 = 1/500,000$.

The truth is that every row with city = Dallas already has state = TX. The state predicate adds no selectivity at all, so the real figure is $1/10,000$ — **fifty times larger than estimated**.

So the optimiser expects two rows and receives a hundred. Expecting two, it chooses an index nested loop; receiving a hundred, that choice is fifty times more expensive than the hash join it rejected. **Extended statistics** — declared on correlated column groups — are the fix in PostgreSQL and SQL Server, and **they must be created explicitly**: no system discovers correlations on its own.

ERRORS COMPOUND MULTIPLICATIVELY

An estimate at a leaf feeds the estimate of the operator above it, which feeds the one above that. The errors multiply rather than averaging out.

A factor of ten at each of four levels is a factor of ten thousand at the top of the plan. This is why deep plans over many tables go wrong far more dramatically than simple ones.

And underestimates are considerably worse than overestimates. An underestimate leads the optimiser to choose a nested loop (which is catastrophic at scale) and to size a hash table too small (which spills, per Module 06). An overestimate leads it to choose a hash join and allocate generously — wasteful, but it degrades gracefully.

Leis et al.'s 'How Good Are Query Optimizers, Really?' measured this on real schemas and found estimates wrong by orders of magnitude as a routine matter, with errors growing sharply with the number of joins. It is the honest picture of where the field stands, and it is worth reading rather than assuming the textbook account describes practice.

- **Table and index cardinalities and page counts.** Cheap to maintain and usually accurate.
- **Histograms**, typically equi-depth, capturing the distribution within a column. These handle skew that the uniformity assumption misses.
- **Distinct value counts**, frequently estimated with HyperLogLog rather than computed exactly, because an exact count requires a sort or a hash of the whole column.
- **Most common values and their frequencies**, stored separately from the histogram. This handles the heavy tail that equi-depth buckets smooth over — a value occurring in 30% of rows deserves its own entry.
- **Correlation statistics — only if you create them.** This is the gap in every system, and it is the gap that §2's example falls into.
- **And all of it is sampled and goes stale.** Running ANALYZE after a bulk load is not optional; a table that the optimiser believes is empty will be planned as though it were.

3 Plan enumeration

SYSTEM R'S DYNAMIC PROGRAMMING, FROM 1979 AND STILL IN USE

The search space is enormous. The number of join orders for n tables is $(2n-2)!/(n-1)!$. At $n = 10$ that is roughly 17 billion orderings, before considering which algorithm to use for each join or which access path for each table.

Dynamic programming builds up by subset size. Find the best plan for every single table, then the best for every pair (using the single-table plans), then every triple (using the pair plans), and so on. **This is $O(3^n)$ rather than factorial** — still exponential, and vastly smaller.

Interesting orders are tracked separately. A plan that is more expensive but produces output sorted on a useful column may win overall, because it saves a sort later (Module 06 §2). So the algorithm keeps the cheapest plan for each distinct useful ordering, not just the cheapest plan overall. **This refinement is what makes sort-merge joins choosable at all.**

Beyond about 10 to 12 tables the exponential cost becomes unacceptable, and systems switch to heuristics — greedy selection, or PostgreSQL's genetic algorithm, which kicks in at twelve tables by default.

	Left-deep plans	Bushy plans
Shape	Every join's right input is a base table, so the tree leans entirely one way.	A join may take two join results as its inputs.
Pipelining	Fully pipelined — one hash table is built at a time, and the rest streams.	Multiple hash tables may be live simultaneously, so memory consumption is higher.
Search space	Much smaller — $n!$ orderings rather than the full count.	Far larger.
Who considers them	System R did, and most OLTP-oriented optimisers still restrict to them.	Analytical systems consider bushy plans , because star and snowflake schemas have genuinely better bushy solutions. The left-deep restriction is a real limitation there.

4 Diagnosing a bad plan

- 1 **Run EXPLAIN ANALYZE** and compare estimated against actual row counts at every node. This single comparison resolves most investigations.
- 2 **Find the deepest node with a large error.** Errors propagate upward, so a wildly wrong estimate at the root is often the consequence of a modest error three levels down. **This is the step people skip**, and fixing the top-level symptom while the real error remains is why tuning sessions go in circles.
- 3 **Ask why that estimate is wrong.** Stale statistics? Correlated predicates? A predicate form the optimiser cannot estimate at all — a function call, a parameter it cannot see, a LIKE pattern?

- 4 **Fix the estimate.** Run ANALYZE; create extended statistics on the correlated columns; rewrite the predicate into a form that can be estimated; or add the column to an index so its distribution is known.
- 5 **Only then consider hints or plan pinning** — and understand that you are freezing a decision that is correct for today's data volume and distribution. **A bad estimate corrects itself when statistics are refreshed; a hint never does.**

Fix the estimate, not the plan. This is the single most useful rule in query tuning, and it is the one most often violated.

WHY OPTIMISERS ARE WORSE THAN YOU WOULD HOPE

Cardinality estimation is unsolved. Fifty years of sustained effort have produced steady improvement and no general solution, because estimating the size of an arbitrary predicate's result over arbitrary correlated data is genuinely hard.

Cost models are approximations of hardware — and often hardware they were calibrated against years ago. The default constants in several systems still encode assumptions about the relative cost of sequential and random I/O that SSDs invalidated.

The search space cannot be explored exhaustively beyond a dozen tables, so heuristics prune it, and the heuristics are sometimes wrong in ways nothing detects.

Learned optimisers — using machine learning for cardinality estimation or plan selection — are an active research area with promising results and are not yet dependable in production, particularly on queries unlike their training data.

So the practical skill is reading plans and fixing estimates, and that is a genuine engineering skill rather than a workaround for an immature system. It will remain necessary.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-445 — Lectures 15–16, Query Planning and Optimization	Rewrites, cost estimation, and System R enumeration.
Selinger et al. — Access Path Selection in a Relational Database Management System (1979, free)	The System R paper. Dynamic programming join enumeration and interesting orders, in the original. Still worth reading.
Leis et al. — How Good Are Query Optimizers, Really? (free)	The measurement study behind §2's honest assessment. Read it.
PostgreSQL — planner statistics and extended statistics	What the optimiser knows and how to give it more, including the correlation fix in §2.

Resource	What it covers for this module
Use The Index, Luke — execution plan chapters	Reading plans across several database systems.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a query with a filter above a join. Use EXPLAIN to confirm the optimiser pushes it below, and construct a case where it cannot.
- 2 Write a correlated subquery over a large table. Check whether your system flattens it, and compare timings against the hand-written join.
- 3 Construct the correlation example from §2 with real data. Report estimated and actual rows, then add extended statistics and report again.
- 4 Build a four-join query and record the estimation error at each level. Confirm the errors compound.
- 5 Delete the statistics on a table and show how the plan changes. Then ANALYZE and show it change back.
- 6 Count the join orders for 5, 10, and 15 tables, and confirm the factorial growth that motivates dynamic programming.
- 7 Construct a query where a more expensive sorted plan wins overall because it avoids a later sort. Confirm with EXPLAIN.
- 8 Force PostgreSQL to use its genetic optimiser by exceeding the join threshold, and compare plan quality and planning time against dynamic programming.
- 9 **Take a genuinely slow query**, apply the five-step method of §4, and document each step and what it revealed.
- 10 Apply a hint to fix a plan, then change the data volume by 100× and show the hint is now wrong.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the two stages of optimisation and say why only one needs a cost model.
- 2 Give six algebraic rewrites and say why each is unconditionally applied.
- 3 Why is subquery flattening so valuable, and when does it fail?
- 4 State the three estimation assumptions and give a formula for each.
- 5 Work through the city/state correlation example with numbers.
- 6 Why do estimation errors compound, and why are underestimates worse?

- 7 Name five kinds of statistics and say which one systems do not maintain automatically.
- 8 Describe System R's enumeration and explain interesting orders.
- 9 Compare left-deep and bushy plans on four axes.
- 10 Give the five steps for diagnosing a bad plan, and the rule that summarises them.